FairShed: fair and auditable load-shedding with Qiskit

Team: [team name]. Members: [names]. Domain: Energy and Smart Infrastructure.
Repository: https://github.com/cheesesocrates/Qiskit-Hack. Demo video: [link].
Simulator only. Synthetic data (real weather snapshot).

1. Summary

When there is not enough electricity, a utility switches whole feeders (the power lines that supply one neighbourhood) off for an hour at a time. FairShed decides which feeders go off in each hour. Hospitals, water pumps and telecom towers are never cut, outage hours are shared fairly, and no area is cut hour after hour. We solve this optimisation problem with Qiskit quantum circuits, run on a simulator, that only ever produce plans that follow the rules. Every quantum result is checked against an exact classical answer, and at this size the classical answer is better, so we claim no speed-up.

- **Every plan is valid.** Both rule-preserving circuits gave a valid plan in 100% of 4096 tries; the standard penalty circuit managed only 27.9%.
- **A fair lottery.** The 4 tied best plans got equal chances (largest difference 4.8e-18).
- **Quantum estimates match exact values.** Counting valid plans: 9.07 against exactly 9. Tail risk (CVaR): 0.911 MW against 0.906 MW.
- **Where quantum loses.** The best circuit gets 85% of the way to the best plan on average and finds it in 0.93% of single tries. Simple round-robin gets 95%; the exact solver gets 100%.

Table 1. Where to find each requirement.

Requirement (hackathon brief)	Rubric line	Section
Identify the problem and its users	A. Problem validity	2
Formulate variables, objective, constraints, test instance	A. Problem validity	2, 3
Justify the quantum algorithm and the mapping	B. Quantum use	3, 4
Implement a working, reproducible Qiskit component	C. Implementation	4, 9
Evaluate against an exact reference	C. Validation	6
Reflect on correctness, limits and trade-offs	B. Awareness of limits	8
Architecture diagram	C. Implementation	5
Originality	D. Innovation	7

2. Problem statement

2.1 Context and users

Pakistan's power shortfall exceeded 4,000 MW in Oct 2026 with 3-4 hour cuts. In Sept 2026 the regulator Nepra found K-Electric's outages on 620 feeders frequently exceeded its own limits and banned loss-based feeder cuts. A federal committee is reviewing a load-shedding exemption for telecom towers; express feeders already protect hospitals and water pumps. [Team: add source links for these facts.]

- **Utility planner:** needs a cut schedule for the next hours that meets the shortfall.
- **Regulator or public:** need to check that a published schedule follows the rules.

2.2 Objective and rules

Objective: choose which feeders are off in each hour so the total cost (harm + unfairness + back-to-back cuts) is as low as possible.

- **Quota:** exactly the required number of feeders is off in each hour.
- **Protection:** hospitals, water pumps and telecom towers are never off.

2.3 Test instance

- 8 synthetic feeders on illustrative Karachi zones, 3 hours.
- Feeders off per hour: 3, 2, 4. Protected: feeder 2 (hospital), feeder 6 (water pump).
- 18 qubits; 4500 valid plans out of 262144 on/off patterns; 4 plans tie for the best cost (10.50).

2.4 How success is measured

Cost (lower is better), the share of tries that are valid plans, the approximation ratio (how close a plan is to the best one: 0 is the worst valid plan, 1 the best, invalid plans score 0) and the chance that one try (one measurement) is a best plan.

3. Formulation

- **Decision variables:** x[t][f] = 1 if feeder f is off in hour t, else 0.
- **Cost:** 1.0 x harm + 1.0 x unfairness + 2.0 x back-to-back cuts. Harm sums a per-feeder, per-hour score over the feeders that are off. Unfairness sums, over feeders, the squared gap between total outage hours (past + new) and the average. Back-to-back counts feeders off in two adjacent hours.
- **Hard rules:** the hourly quota and the protected feeders.

3.1 From plan to qubits

- One qubit per (hour, feeder) switch, qubit q = t x N + f; protected feeders get no qubit, so they can never be switched off.
- The cost is a QUBO (a quadratic function of 0/1 variables). Writing x = (1 - Z)/2 turns it into Z and ZZ terms, which become RZ and RZZ rotation gates.
- The quota is not a penalty: it is built into the circuit (Section 4).

4. Quantum approach and why

4.1 Key idea

We use QAOA: a short quantum circuit that alternates a cost step and a mixing step, with its angles tuned by a classical optimiser. Each hour starts in a Dicke state, an equal mix of every way to switch off exactly the required number of feeders. The mixer (the step that moves probability between plans) never changes how many feeders are off. So every measured plan is valid by construction. The standard penalty version only discourages invalid plans and wastes most of its tries on them.

Table 2. Quantum components.

Component	What it does	Why this one
Dicke + XY mixer QAOA	Swaps which feeders are off, never how many	Rules hold in every sample
Grover mixer QAOA	Treats all valid plans alike	Equal chances for tied plans; picks the dashboard and replay plans
Penalty QAOA	Rules only as a cost penalty	Weak baseline
Counting (amplitude estimation)	Estimates the share of valid plans	Planner freedom, checkable
Tail risk (amplitude estimation)	Estimates expected loss and CVaR from forecast errors	Exact check available
Grover Adaptive Search	Repeated search for a better plan	Second method to compare
MPS scaling	Same circuit, larger cities, compressed simulator	Behaviour beyond exact simulation

4.2 Implementation and assumptions

Qiskit 2.6.2 circuits in src/quantum/ run with scripts/run_quantum.py on the Qiskit Aer 0.18.2 simulator. Amplitude estimation means estimating a probability that a circuit encodes. Assumptions: an ideal simulator (no noise, no hardware), results from a finite number of measurements, angles tuned by the classical optimiser COBYLA, and a plan is the best valid result of 256 measurements.

5. Architecture

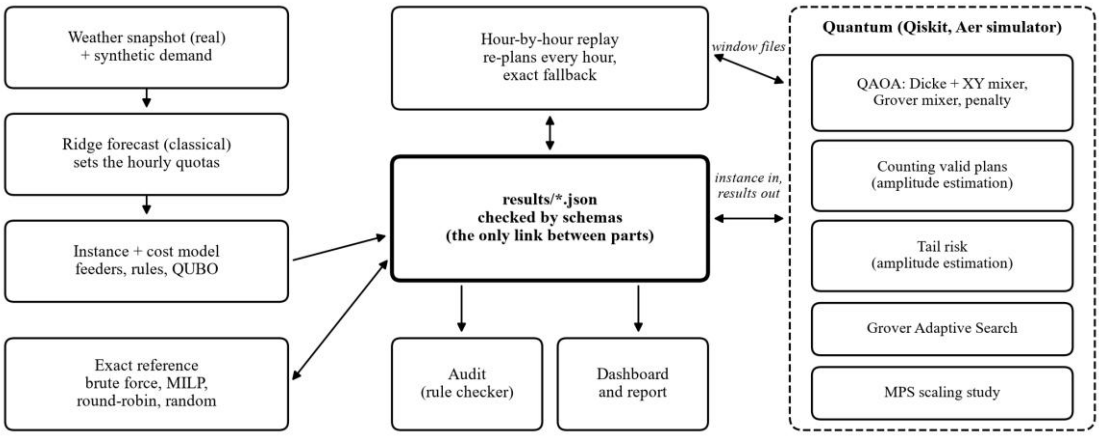


Figure 1. Architecture: all parts exchange only JSON files.

A classical ridge-regression forecast, trained on a real weather snapshot and synthetic demand, sets how many feeders must be off each hour. The quantum and classical parts never call each other's code: they exchange only JSON files, each checked against a schema. The hour-by-hour replay asks the quantum solver for a plan through files and falls back to the exact solver if it fails.

6. Results against the classical reference

6.1 Base instance

Table 3. Base instance, 18 qubits. Circuits: 3 layers, 4096 measurements each.

Method	Valid plans (%)	Chance one try is best	Approximation ratio
Exact (brute force; MILP agrees)	100.0	1.0000	1.000
Round-robin (take turns)	100.0	0.0000	0.952
Random valid plan	100.0	0.0000	0.619
Dicke + XY mixer QAOA	100.0	0.0059	0.798
Grover mixer QAOA	100.0	0.0093	0.846
Penalty QAOA	27.9	0.0002	0.183

Both rule-preserving circuits are always valid; the penalty circuit is valid in 27.9% of tries. On average quality, round-robin (0.952) and the exact solver (1.000) beat the best circuit (0.846).

6.2 Other quantum components

Table 4. Other quantum components against their exact value.

Component	Quantum result	Exact value	Agrees
Counting (4x2 block)	9.07 (8.90 to 9.23)	9	yes
Tail risk, CVaR 95%	0.911 MW	0.906 MW	yes
4 tied best plans	spread 4.8e-18	equal	yes
Grover search (4x2 block)	90% valid, 30% best	100% best	worse
Fairness-weight sweep	5 of 6 at best cost	6 of 6	mostly
Replay (12 of 12 hours by circuit)	9 best, 12 pass rules	12 best	mostly
MPS scaling	52 of 52 runs, up to 72 qubits	MILP	see 6.3

The amplitude-estimation results agree with the exact values; the search and planning results reach the best plan only part of the time.

6.3 Rules under stress and larger cities

Heatwave scenario (no feeder off more than 1 hour in a row): round-robin breaks the rules 4 times and 99.6% of random valid plans break the back-to-back limit. The exact plan passes every rule.

An MPS (matrix product state) simulator compresses the quantum state; the bond dimension sets how much it keeps. At 36 qubits, bond dimension 64 gave 98% valid plans (ratio 0.73) in 3.3 MB, where exact simulation needs 1.1 TB. At 72 qubits the best run (bond dimension 32) gave 99.8% valid plans (ratio 0.81) but bond dimension 64 gave 80%, so the result has not settled. Fidelity (overlap with the exact state) is measurable only up to 20 qubits: 0.94 at bond dimension 64. This is a classical simulation, not a quantum advantage.

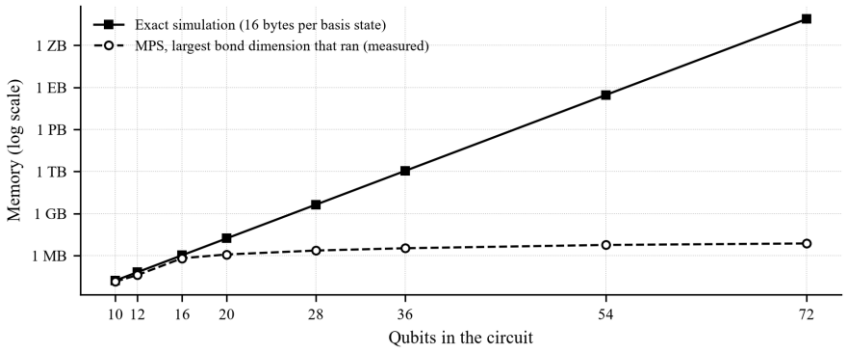


Figure 2. Memory for exact simulation (computed) and for the MPS simulation (measured).

7. What is original

- **Fairness in the model:** past outage hours and back-to-back cuts count, not only harm.
- **Rules in the circuit:** every sample is a valid plan, with no penalty tuning.
- **A fair lottery:** tied best plans get exactly equal chances.
- **Audit and replay:** any schedule can be checked; the replay re-plans hourly with an exact fallback.

8. Limitations and trade-offs

- **Simulator and synthetic data:** an ideal, noise-free simulator, no hardware; feeders, demand and outage history are made up, only the weather snapshot is real.
- **Classical wins at this size:** the exact solver is better and faster; no speed-up is claimed.
- **Low single-try chance:** one try is a best plan only 0.93% of the time, so a plan is the best of 256 tries; this still missed the best plan at 1 of 6 fairness weights and 3 of 12 replay hours.
- **MPS limits:** Aer crashed on Windows on some circuits at bond dimension 2 or 4 (1 run retried); fidelity is measurable only up to 20 qubits; angles from small instances are reused; the largest instances do not settle.
- **Risk block:** 8 feeders, not 4 to 6, because smaller blocks have zero loss in every scenario.

9. Reproducibility

- Python 3.11, qiskit 2.6.2, qiskit-aer 0.18.2, qiskit-algorithms 0.4.0, qiskit-optimization 0.7.0 (requirements.txt); seed 7.
- Commands: python scripts/tasks.py setup, then test, demo and ui (the dashboard). Code and results: https://github.com/cheesesocrates/Qiskit-Hack.
- 36 of 36 proof checks pass (results/checks.json); scripts/validate_results.py checks every results file against its schema. Optimiser angles differ slightly between machines, so sampled counts reproduce on one machine.